



UNIVERSIDADE
DE ÉVORA

Project - Practical Part

**Cross-Site Scripting (XSS): real impact and
effective mitigation**

Segurança em Sistemas Informáticos

Done by:

Tiago Pinto m68106
Mestrado em Engenharia Informática

Introduction

Objective

The objective of this project is to implement an experimental environment designed to demonstrate the impact of Cross-Site Scripting (XSS) vulnerabilities and evaluate the effectiveness of different mitigation mechanisms.

Four independent web applications were developed using the Flask framework, allowing the demonstration of:

- Stored XSS exploitation;
- Mitigation through Output Encoding;
- Mitigation through Content Security Policy (CSP);
- Cookie protection using the HttpOnly attribute.

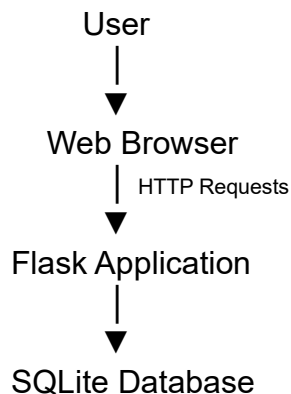
The implementation aims to complement the theoretical analysis presented in the main paper by providing practical evidence of how XSS vulnerabilities can be exploited and how modern security mechanisms can mitigate them.

Technologies Used

- **Python 3:** Main programming language;
- **Flask:** Web framework;
- **SQLite:** Comment storage;
- **HTML5:** Web interface;
- **JavaScript:** Attack demonstration;
- **Mozilla Firefox:** Testing environment.

System Architecture

The implemented architecture follows the traditional client-server model.



The user interacts with the application through a web browser. Requests are processed by the Flask server, which communicates with an SQLite database to store and retrieve information.

Implementation of the Vulnerable Scenario

Objective

The objective of this scenario is to demonstrate a Stored XSS vulnerability. In this implementation, user comments are stored without any validation or sanitization mechanism.

Implementation Details

The user-submitted comment is directly stored in the database:

```
comment = request.form["comment"]
```

The stored content is later displayed using:

```
{{ comment[0] | safe }}
```

The **safe** filter instructs the Jinja2 template engine to render the content as raw HTML.

Attack Execution

The following payload was used:

```
<script>alert("XSS")</script>
```

After submission, the malicious script is stored in the database and executed every time the page is loaded.

Results

The JavaScript code was executed automatically when the page was rendered. This behavior confirms the presence of a Stored XSS vulnerability.



Figure 1. Successful execution of the stored XSS payload.

Mitigation Through Output Encoding

Objective

The purpose of this phase is to prevent the execution of malicious JavaScript code.

Implementation

The **escape()** function provided by the MarkupSafe library was used:

```
comment = escape(request.form["comment"])
```

This function converts special characters into safe HTML entities.

Operation

For example, the string:

```
<script>
```

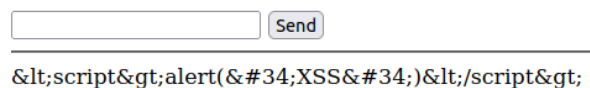
is transformed into:

```
&lt;script&gt;
```

As a result, the browser interprets the content as plain text rather than executable code.

Results

After implementing output encoding, the payload was displayed as text and no longer executed.



<script>alert("XSS")</script>

Figure 2. Malicious payload displayed as text after output encoding was applied.

Mitigation Through Content Security Policy

Objective

The objective of this experiment is to demonstrate how browser-enforced security policies can reduce the impact of XSS attacks.

Implementation

The following HTTP response header was configured:

```
response.headers["Content-Security-Policy"] = (  
    "default-src 'self'; "  
    "script-src 'self'; "  
    "object-src 'none'; "  
    "base-uri 'self';"  
)
```

Operation

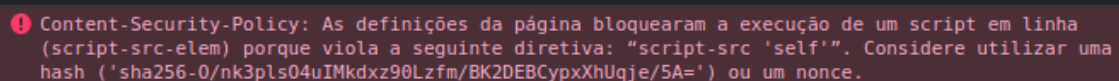
The Content Security Policy restricts the execution of unauthorized scripts and limits the sources from which active content may be loaded. This significantly reduces the attack surface available to malicious actors.

Results

When attempting to execute:

```
<script>alert("CSP TEST")</script>
```

the browser blocked the script execution.



Content-Security-Policy: As definições da página bloquearam a execução de um script em linha (script-src-elem) porque viola a seguinte diretiva: "script-src 'self'". Considere utilizar uma hash ('sha256-0/nk3pls04uIMkdxz90Lzfm/BK2DEBCypxXhUqje/5A=') ou um nonce.

Figure 3. Inline script blocked by the Content Security Policy.

Cookie Protection Using HttpOnly

Objective

The purpose of this experiment is to demonstrate how the HttpOnly attribute reduces the impact of XSS attacks by protecting session cookies.

Implementation

Secure Cookie:

```
response.set_cookie(  
    "sessionid",  
    "ABC123",  
    httponly=True  
)
```

Insecure Cookie:

```
response.set_cookie(  
    "sessionid",  
    "ABC123"  
)
```

Operation

When the HttpOnly attribute is enabled, JavaScript code running in the browser cannot access the cookie through **document.cookie**. This prevents attackers from stealing session cookies through XSS attacks.

Results

In the insecure scenario:

```
document.cookie
```

returned:

```
sessionid=ABC123
```

In the protected scenario:

```
document.cookie
```

did not expose the cookie value.

```
>> document.cookie
< ""
```

Figure 4. Cookie protected by the HttpOnly attribute.

```
>> document.cookie
< "sessionId=ABC123"
```

Figure 5. Cookie accessible through JavaScript in the insecure implementation.

Comparative Evaluation

Results Summary

Scenario	XSS Execution	Cookie Theft
Vulnerable Application	Yes	Yes
Output Encoding	No	No
Content Security Policy	No	Partially
HttpOnly	Yes	No
Combined Mitigations	No	No

Discussion

The results demonstrate that no single mitigation technique provides complete protection against XSS attacks.

While HttpOnly successfully prevents cookie theft, it does not stop malicious JavaScript execution.

Similarly, CSP significantly reduces the attack surface but relies on proper configuration and deployment.

The most effective approach is a layered security strategy combining:

- Output Encoding;
- Content Security Policy;
- HttpOnly Cookies;
- Secure Development Practices.

This defense-in-depth approach aligns with current industry recommendations.

Limitations

The developed environment presents several limitations:

- Simplified laboratory environment;
- SQLite used instead of a production-grade database;
- No real authentication mechanism;
- No concurrent users;
- Tests performed exclusively in a local environment.

Despite these limitations, the implemented system accurately reproduces the fundamental concepts associated with Cross-Site Scripting attacks and their mitigation.

Conclusion

This practical project successfully demonstrated the real impact of Cross-Site Scripting vulnerabilities in web applications. The experiments showed how malicious JavaScript can be executed when user input is processed without proper security controls.

The implementation and evaluation of multiple mitigation techniques demonstrated that effective protection against XSS requires a layered security approach. The results obtained confirm the recommendations provided by security organizations.

By combining Output Encoding, Content Security Policy, and HttpOnly cookies, developers can significantly reduce the risk posed by XSS attacks and improve the overall security posture of web applications.